
Gestión de vistas y subconsultas en MySQL

Introducción

Las vistas y subconsultas son herramientas fundamentales en los sistemas de gestión de bases de datos relacionales como MySQL. Permiten estructurar consultas complejas, reutilizar lógica y mejorar tanto la seguridad como el rendimiento cuando se utilizan correctamente.

Vistas en MySQL

Una vista es una tabla virtual basada en el conjunto de resultados de una consulta SQL. Las vistas contienen filas y columnas, igual que una tabla real, pero no almacenan físicamente los datos (con algunas excepciones que veremos en la sección de materialización).

Creación de vistas

La sintaxis básica para crear una vista en MySQL es:

```
CREATE [OR REPLACE] VIEW nombre_vista [(columna1, columna2, ...)]
AS consulta_select
[WITH [CASCADED | LOCAL] CHECK OPTION];
```

Ejemplo:

```
CREATE VIEW empleados_departamento AS
SELECT e.id, e.nombre, e.apellido, d.nombre_departamento
FROM empleados e
JOIN departamentos d ON e.departamento_id = d.id;
```

Una vez creada, podemos consultar la vista como si fuera una tabla normal:

```
SELECT * FROM empleados_departamento;
```

Modificación y eliminación de vistas

Para modificar una vista existente:

```
ALTER VIEW nombre_vista AS nueva_consulta;
```

O utilizando *CREATE OR REPLACE VIEW*:

```
CREATE OR REPLACE VIEW empleados_departamento AS
SELECT e.id, e.nombre, e.apellido, d.nombre_departamento, e.fecha_contratacion
FROM empleados e
JOIN departamentos d ON e.departamento_id = d.id;
```

Para eliminar una vista:

```
DROP VIEW IF EXISTS nombre_vista;
```

Vistas actualizables

Una vista es actualizable (se pueden realizar operaciones INSERT, UPDATE, DELETE) si cumple con ciertas condiciones:

1. La vista debe estar basada en una única tabla
2. No debe contener funciones de agregación (SUM, COUNT, AVG, etc.)
3. No debe contener DISTINCT, GROUP BY o HAVING
4. No debe contener subconsultas en la cláusula SELECT o WHERE
5. No debe utilizar UNION, UNION ALL u otras operaciones de conjuntos

Ejemplo de vista actualizable:

```
CREATE VIEW ingenieros AS
SELECT id, nombre, apellido, email, salario
FROM empleados
WHERE departamento_id = 3;
```

Ahora podemos insertar, actualizar o eliminar registros a través de esta vista:

```
INSERT INTO ingenieros (nombre, apellido, email, salario)
VALUES ('Juan', 'Pérez', 'jperez@empresa.com', 60000);
```

```
UPDATE ingenieros SET salario = 65000 WHERE id = 101;
```

```
DELETE FROM ingenieros WHERE id = 102;
```

Limitaciones de las vistas

Si intentamos actualizar una vista no actualizable, MySQL generará un error. Para controlar este comportamiento, podemos usar CHECK OPTION:

```
CREATE VIEW ingenieros_senior AS
SELECT id, nombre, apellido, email, salario
FROM empleados
WHERE departamento_id = 3 AND salario > 50000
WITH CHECK OPTION;
```

Con **CHECK OPTION**, MySQL verifica que cualquier operación de inserción o actualización cumpla con la condición **WHERE** de la vista. Por ejemplo, no permitirá insertar un ingeniero con salario menor a 50000.

Subconsultas en MySQL

Una subconsulta es una consulta SQL anidada dentro de otra consulta. Pueden utilizarse en varias partes de una consulta SQL, como las cláusulas **SELECT**, **FROM** y **WHERE**.

Tipos de subconsultas

1. **Subconsultas escalares:** Devuelven un único valor.

```
SELECT nombre, apellido, salario,
       (SELECT AVG(salario) FROM empleados) AS salario_promedio
FROM empleados;
```

2. **Subconsultas de fila:** Devuelve una sola fila con varias columnas.

```
SELECT *
FROM empleados
WHERE (departamento_id, salario) = (
    SELECT departamento_id, MAX(salario)
    FROM empleados
    GROUP BY departamento_id
    HAVING departamento_id = 3
);
```

3. **Subconsultas de columna:** Devuelven una sola columna con múltiples filas.

```
SELECT *
FROM empleados
WHERE departamento_id IN (
    SELECT id
    FROM departamentos
    WHERE ubicacion = 'Rosario'
);
```

4. **Subconsultas de tabla:** Devuelve múltiples filas y columnas.

```
SELECT e.nombre, d.nombre_departamento
FROM empleados e
JOIN (
    SELECT id, nombre_departamento, COUNT(*) as num_empleados
    FROM departamentos
    JOIN empleados ON departamentos.id = empleados.departamento_id
    GROUP BY departamentos.id
    HAVING COUNT(*) > 10
) d ON e.departamento_id = d.id;
```

Subconsultas correlacionadas

Una subconsulta correlacionada hace referencia a columnas de la consulta externa. Se ejecuta una vez por cada fila procesada por la consulta externa.

```
SELECT e.nombre, e.apellido, e.salario
FROM empleados e
WHERE e.salario > (
    SELECT AVG(salario)
    FROM empleados
    WHERE departamento_id = e.departamento_id
);
```

Esta consulta encuentra empleados cuyo salario es mayor que el promedio en su propio departamento.

Operadores para subconsultas

IN / NOT IN: Comprueba si un valor está/no está en el conjunto de resultados de la subconsulta.

```
SELECT *
FROM empleados
WHERE departamento_id IN (
    SELECT id
    FROM departamentos
    WHERE presupuesto > 1000000
);
```

ANY / SOME: Comprueba si la condición se cumple para algún valor de la subconsulta.

```

SELECT *
FROM empleados
WHERE salario > ANY (
    SELECT salario
    FROM empleados
    WHERE departamento_id = 3
);

```

ALL: Comprueba si la condición se cumple para todos los valores de la subconsulta.

```

SELECT *
FROM empleados
WHERE salario > ALL (
    SELECT salario
    FROM empleados
    WHERE departamento_id = 2
);

```

EXISTS / NOT EXISTS: Comprueba si la subconsulta devuelve algún resultado.

```

SELECT *
FROM departamentos d
WHERE EXISTS (
    SELECT 1
    FROM empleados e
    WHERE e.departamento_id = d.id
    AND e.salario > 80000
);

```

Materialización

Concepto de materialización

Una vista materializada es el resultado pre-calculado de una consulta. A diferencia de una simple vista, el resultado de la vista materializada está guardado en algún lado, generalmente en una tabla. Las vistas materializadas se utilizan cuando se necesita una respuesta inmediata y la consulta en la que se basa tarda demasiado en generar un resultado. Las vistas materializadas deben actualizarse periódicamente. La frecuencia de actualización y la actualidad de su contenido dependen de los requerimientos. Básicamente, una Vista Materializada puede actualizarse de forma inmediata o diferida, completamente o hasta un momento determinado.

Vistas materializadas en MySQL

A diferencia de otros SGBD como Oracle o PostgreSQL, MySQL no tiene un soporte nativo para vistas materializadas. Sin embargo, existen varias técnicas para implementarlas.

Implementación de vistas materializadas

Un breve ejemplo de cómo se podría hacer esto es la siguiente consulta:

```
SELECCIONAR *)  
DE mi_tabla_conpocosdatos;
```

Devuelve un resultado inmediato porque la tabla no tiene muchos datos. La siguiente consulta puede tardar desde segundos hasta minutos:

```
SELECCIONAR *  
DE mi_tabla_pesada_conmillonesdefilas;
```

Una posible solución a esto es crear una tabla aparte y almacenar los datos que necesitemos.

Dependiendo de las necesidades que tengamos, podemos actualizar esta tabla una vez al día, cada hora o en casos extremos, cada vez que se haga una actualización en la tabla original.

Tablas derivadas con actualización programada:


```

-- creacion de la tabla materializada
CREATE TABLE ventas_por_region (
    region VARCHAR(50),
    total_ventas DECIMAL(10,2),
    fecha_actualizacion TIMESTAMP,
    PRIMARY KEY (region)
);

-- procedimiento para actualizar la tabla materializada
DELIMITER //
CREATE PROCEDURE actualizar_ventas_por_region()
BEGIN
    -- vaciar la tabla
    TRUNCATE TABLE ventas_por_region;

    -- insertar datos actualizados
    INSERT INTO ventas_por_region (region, total_ventas, fecha_actualizacion)
    SELECT r.nombre, SUM(v.monto), NOW()
    FROM ventas v
    JOIN regiones r ON v.region_id = r.id
    GROUP BY r.nombre;
END //
DELIMITER ;

-- crear un evento para actualizar periodicamente (cada dia a la 1am)
CREATE EVENT actualizar_ventas_diario
ON SCHEDULE EVERY 1 DAY
STARTS '2025-03-13 01:00:00'
DO
    CALL actualizar_ventas_por_region();

```

Actualización bajo demanda con verificación de tiempo:

```

DELIMITER //
CREATE PROCEDURE obtener_ventas_por_region()
BEGIN
    -- verificar si los datos estan desactualizados (+24hrs)
    IF (SELECT MAX(fecha_actualizacion) FROM ventas_por_region) < NOW() - INTERVAL 1 DAY THEN
        CALL actualizar_ventas_por_region();
    END IF;

    -- devolver datos
    SELECT region, total_ventas FROM ventas_por_region;
END //
DELIMITER ;

```

Triggers para mantener la sincronización:

```

DELIMITER //
CREATE TRIGGER after_venta_insert
AFTER INSERT ON ventas
FOR EACH ROW
BEGIN
    DECLARE region_nombre VARCHAR(50);

    -- obtener el nombre de la región
    SELECT nombre INTO region_nombre FROM regiones WHERE id = NEW.region_id;

    -- actualizar tabla materializada
    INSERT INTO ventas_por_region (region, total_ventas, fecha_actualizacion)
    VALUES (region_nombre, NEW.monto, NOW())
    ON DUPLICATE KEY UPDATE
        total_ventas = total_ventas + NEW.monto,
        fecha_actualizacion = NOW();
END //
DELIMITER ;

-- también necesitaríamos triggers para el UPDATE y el DELETE

```

Estas implementaciones tienen ventajas y desventajas, y la elección depende de:

- Frecuencia de consulta vs. frecuencia de actualización
- Complejidad de la consulta original
- Requisitos de consistencia

Referencias y lecturas adicionales

- [Documentación oficial de MySQL sobre vistas](#)
- [Documentación oficial de MySQL sobre subconsultas](#)
- [MySQL Performance Schema](#)